Métodos de Optimización

Clase 3 (complemento): ¿tuning o tanteo?

[Nombre del profesor]

Septiembre de 2026

Programa de Maestría, [Nombre de la universidad / programa]

De dónde sale esto

Los ejemplos manuales 1D de basin hopping, simulated annealing y PSO (material complementario de la Clase 3) sólo encuentran el mínimo global de forma confiable **después** de ajustar T, el `step_size` y el peso de inercia w.

La pregunta incómoda

Para llegar a esos valores corrí muchas combinaciones de hiperparámetros sobre varias semillas hasta quedarme con una que funcionara bien. *¿Eso no es simplemente tanteo?*

Respuesta corta: en parte sí, en parte no – y la distinción importa mucho para usar estos métodos de forma responsable. Estas slides muestran el procedimiento completo, con datos.

Dos cosas que se ven parecidas pero no lo son

Tuning sistemático

- Fijar una combinación de hiperparámetros.
- Correr **muchas semillas** distintas.
- Medir la **tasa de éxito** (¿qué fracción llegó al global?).
- Elegir la combinación con mejor tasa.

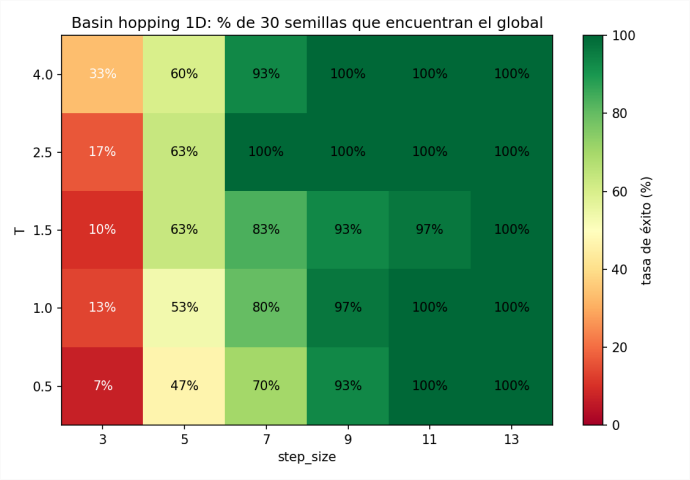
Cherry-picking (tanteo)

- Correr con una semilla.
- Si no funciona, cambiar algo al azar.
- Repetir hasta que **una** corrida se vea bien.
- Publicar esa corrida como *el resultado*.

Lo que se hizo aquí

Un barrido de hiperparámetros (tuning sistemático) para elegir la *región* de $(T, \text{step_size})$; la semilla usada en cada figura (`seed=0`) fue la primera que se probó **dentro de esa región ya confiable** – no una semilla especial elegida por su resultado.

El barrido real: basin hopping 1D



Lo que dice el mapa de calor

- `step_size` importa **mucho más** que T en este paisaje: con `step_size`≥ 9 la tasa de éxito es $\geq 93\,\%$ para **cualquier** T probado; con `step_size`$= 3$ nunca supera el $33\,\%$, sin importar T.
- Esto tiene sentido: el mínimo global está a $\sim$15 unidades del punto de arranque, y con saltos chicos casi nunca se cruza esa distancia en 30 iteraciones.
- **No hay una fórmula** que diga de antemano *step_size*$= 9$ *es el correcto* – eso solo se supo *después* de correr el barrido.

Con esos parámetros, ¿cualquier semilla funciona?

```
Basin hopping (step_size=9, T=1.5):      28/30 semillas (93%)
Simulated annealing (paso=3.5, T0=6):    28/30 semillas (93%)
Particle swarm (w: 0.9->0.4, N=20):      28/30 semillas (93%)
```

Sí: una vez encontrada la región correcta, la elección de semilla ya casi no importa. Eso es lo que hace *tuning*, no *cherry-picking*.

El verdadero talón de Aquiles

El barrido necesitó algo que en la práctica no se tiene

Para medir *tasa de éxito* hay que saber **dónde está el mínimo global** de antemano. En este ejemplo de clase lo sabemos (diseñamos la función). En un problema real de optimización se usa una metaheurística **precisamente porque no se conoce el paisaje** – no hay vara de medir contra la cual calibrar T o `step_size`.

¿Cómo se tunea algo cuando no se puede verificar si funcionó?

Qué se hace en la práctica, sin conocer la respuesta

- **Diagnósticos relativos**: correr varias semillas y ver si *convergen al mismo punto*. Coincidencia = buena señal (aunque no prueba que sea el global); resultados muy distintos entre semillas = señal de que faltan iteraciones o los hiperparámetros están mal.
- **Heurísticas basadas en la escala del problema**: `step_size` como una fracción del rango de las variables; T_0 elegido para que la tasa de aceptación inicial sea $\sim$20–40 % (se estima con una muestra pequeña de ΔE típicos antes de correr todo el algoritmo).
- **Esquemas autoadaptativos**: ajustar `step_size` solo, según la tasa de aceptación reciente (lo hace `scipy.optimize.basinhopping` internamente); mutación autoadaptativa en algoritmos genéticos; CMA-ES.
- **Aceptar la incertidumbre como parte del resultado**: reportar *mejor de N corridas independientes* en vez de *la respuesta* – lo que de hecho hacen la mayoría de los papers que usan estos métodos.

Síntesis

El ajuste de hiperparámetros que ven en los scripts 1D **sí** fue tuning sistemático (barrido + tasa de éxito sobre muchas semillas), no una semilla con suerte. Pero ese tuning se apoyó en conocer el mínimo global de antemano – un lujo que un problema real casi nunca da.

La lección honesta

Ninguna metaheurística viene con hiperparámetros *correctos* de fábrica. Ajustarlos bien, sin hacer trampa, es en sí mismo parte del trabajo de optimizar – y por eso en la práctica se combina con diagnósticos relativos, heurísticas de escala y reportar resultados de múltiples corridas en vez de una sola.

Hay un resultado teórico (Geman & Geman, 1984) que garantiza convergencia al global con un enfriamiento *logarítmico* $T_k = T_0/\log(k+2)$ – pero es tan lento que nadie lo usa en la práctica; es la prueba de que existe una solución *perfecta* a este problema, solo que es inútil computacionalmente.

Referencias

- S. Geman, D. Geman. *Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images*. IEEE Trans. Pattern Analysis and Machine Intelligence, 6(6), 1984. (Convergencia garantizada de SA con enfriamiento logarítmico.)
- N. Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv:1604.00772, 2016.
- M. Kochenderfer, T. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.
- Scripts de referencia: `scripts/07_basin_hopping_1d.py`, `08_simulated_annealing_1d.py`, `09_particle_swarm_1d.py`, `10_barrido_hiperparametros_1d.py`.

¿Preguntas?